

20/09/2025

Task-4.1

Aim: To use various data types, list, tuples and dictionary in python programming

Algorithm:

~~1. Add elements + Add elements to the list.~~

1. start
2. create an empty list sales = []
3. For 7 days append integer sales to the list using append()
4. Compute total = sum(sales) and avg = total/7
5. Find max-val = max(sales), min-val = min(sales)
6. Find corresponding days with index() (add +1 to convert to day number).

Program:

List scenario

```
days = 7
sales = []
target = 500 # target sales for the day
for s in range(8)
    sample-entries = int(input("Enter the seven days sales count"))
    sales.append(sample-entries) # list.append()
total = sum(sales)
avg = total / days
max-val = max(sales)
min-val = min(sales)
best-day = sales.index(max-val) + 1 # list.index()
worst-day = sales.index(min-val) + 1
print("sales (mon-sun):", sales)
print("total:", total)
print("Average:", round(avg, 2))
print("Best day", best-day, "with", max-val)
print("worst day:", worst-day, "with", min-val)
```


Task-4.2

Aim: To manage and Query an immutable daily lab slot Schedule using a tuple, demonstrating membership checks `count()`, `index()`, and slicing.

Algorithm

1. start
2. Define slots as a fixed tuple of integers.
3. Read Query hour.
4. check existence with Query in slots.
5. use `count()`: if positive, use `index()` to find the best first position.
6. print result.
7. stop

Program

```
# Tuple scenario
```

```
slots = (9, 11, 14, 16, 14) # imm.
```

```
Query = 14
```

```
exists = (Query in slots)
```

```
freq = slots.count(Query) # tuple.count()
```

```
first_pos = slots.index(Query) + 1
```

```
morning = slots[:2]
```

```
afternoon = slots[2:]
```

```
print("All lab slots:", slots)
```

```
print(f"Is {Query}; oo present?", exists)
```

```
print(f"{Query}; oo occurs", freq, "time(s)")
```

```
print("First Occurrence position (1-based):",
```

```
print("Morning slots", morning)
```

```
print("Afternoon slots:", afternoon)
```


Task-4.3

Aim: To manage a live price list and bill a customer using dictionary method and views.

Algorithm:

- 1) start
- 2) create an empty dictionary prices
- 3) Get an item name
- 4) Get the item price
- 5) Add the item and price to prices.
- 6) Ask the user for an item to update.
- 7) Ask the user for an item to remove
- 8) If given remove that item from prices
- 9) stop.

Program:

```
prices = {}  
n1 = int(  
for i in range(n1):  
    item = input("Enter item name:")  
    price = float(input(f"Enter price of {item}:"))  
    prices[item] = price.  
#optional price revision  
rev_item = input("Enter item to update price")  
if rev_item in prices:  
    new_price = float(input(f"Enter new price"))  
    prices.update({rev_item: new_price})  
## Find cost list item.  
cost_list = item = None  
max_price = 0  
for item, price in items():  
    if price > max_price:  
        max_price = price  
        cost_list = item.  
## Remove out of stock item.  
remove_item = input("Enter an item to remove from  
price list for  
removed = price = None
```



```
if remove-item;  
    removed-price = prices.pop(remove-item, none)  
# Display results.
```

```
print("\n Available items:", list(prices.keys()))
```

```
print(" prices:", list(prices.values()))
```

```
if cost list-item
```

```
    print(" cost list item:", cost list-item, "at".
```

```
if remove-item:
```

```
print(F" Removed '{remove-item}' price (if existed):",
```

```
# set operations
```

wrong

removed-price

```
both = ai-hackathon.intersection(robotics-challenge)
```

```
only-ai = ai-hackathon.difference(robotics-challenge)
```

```
only-robotics = robotics-challenge.difference(ai-hackathon)
```

```
unique-all = ai-hackathon.union(robotics-challenge)
```

wrong

Task- 4.4 set-tech Fest participation.

Two events, AI Hackathon and Robotics challenge, have participants IDs stored in two sets. Add a late registrant to AI Hackathon, remove a withdrawn participant from Robotics using `discard()`, then find participants in both events only in one difference, the total unique participants (union).

Get AI Hackathon participants.

ai-hackathon = set()

n1 = int

for in range(n1)

pid = input("Enter participant ID:")

ai-hackathon.add(pid)

Get Robotics challenge participants

robotics-challenge = set()

n2 = int(input)

for in range(n2)

pid = input("Enter participant ID:")

robotics-challenge.add(pid)

Add a late registrant

late-id = input("Enter late registrant ID for AI Hackathon process")

if late-id

ai-hackathon.add(late-id)

Remove a withdrawn participant

remove-id input

Challenge (or press enter to skip):

if remove-id;

robotics-challenge.discard(remove-id)

set operations

both = ai-hackathon.intersection

only-ai = ai-hackathon.difference

only-robotics = robotics-challenge.difference

unique-all = ai-hackathon.union

VEL TECH	
PERFORMANCE	60
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	15
TH DATE	

output:

no. of participants in AI Hackathon: 4

participant ID: A1

participant ID: A2

participant ID: A4

.. " ID: A5

Robotics challenge: 4

Hackathon: {A1, A4, A5, A8, A2}

Robotics: {A1, A7, A3}

Both events: {A2}

unique participants: 7

